



Session 9

By: Prof. Raksha Sharma

Module 2, Week 4
May-23 2026

Numerical Computing with NumPy



Module 2: The Data Handling Toolkit: NumPy and Pandas

Understanding vectorization for high-speed numerical operations; Creating and manipulating NumPy arrays

NumPy; NumPy Arrays; Vectorization; Numerical operations

What and why NumPy



Whenever we need to perform the same algebraic operation on a set of numbers

- Add bonus marks to all students
- Apply GST/tax to all product prices
- Convert temperatures for many sensor readings
- Calculate revenue from price and quantity

NumPy (**Numerical Python**) is a Python library used for storing data and applying **computing**.

NumPy: Key Features



- Have support for `ndarray` (multi-dimensional array object)
- Vectorized operations (faster than loops on Array/List)
- Linear algebra support
- Random number generation of any length
- Integration with libraries like Pandas, Matplotlib, and TensorFlow

numPy Arrays: a data structure to store data

Import numpy as np

#1-D array (or row/column vector)

```
student_marks = np.array([72, 85, 64, 90, 58])
```

#2-D array

```
score_table = np.array([  
    [72, 80, 75],  
    [85, 78, 82],  
    [64, 70, 68],  
    [90, 88, 92],  
    [58, 60, 62]  
])
```

A Python list and numPy array



- Add 5 bonus marks to each student

```
marks_list = [45, 67, 82, 39, 90]
bonus_marks_list = []
for mark in marks_list:
    bonus_marks_list.append(mark + 5)
```

```
# NumPy vectorized approach:
marks_array = np.array([45, 67, 82, 39, 90])
bonus_marks_array = marks_array + 5
```

numPy Array: initialization of huge vectors

Import numpy as np

#1-D array (or row/column vector)

```
zeros_array = np.zeros(5)
```

```
ones_array = np.ones(5)
```

```
numbers = np.arange(1, 11)
```

Create evenly spaced array between a range: Significant while creating plots

```
np.linspace(start, stop, number_of_values)
```

```
np.linspace(0, 1, 5)
```

Output: [0 0.25 0.5 0.75 1]

numPy Array: initialization of huge vectors



Create random number vectors: Significant while running ML algorithms

```
np.random.seed(42)
```

```
random_marks = np.random.randint(35, 101,  
size=10)
```

#seed helps to produce the same 10 number every time you run the experiments

numPy Array: Properties



NumPy array has important properties such as *shape*, *dimension*, *size*, and *data type*.

These describe how the array is organized and stored. numPy has objects to see these properties.

- Shape tells how many rows and columns the array has - 2×3
- Dimension tells how many axes the array has - 2D, 1D, 3D
- Size means the total number of elements - $2 \times 3 = 6$
- Data type tells what kind of values are stored

Common NumPy data types:

- `int32`, `int64` → integers
- `float32`, `float64` → decimal numbers
- `bool` → True/False
- `str` → strings

numPy Array: Properties and objects



- `print('Number of dimensions:',
score_table.ndim)`
- `print('Shape:', score_table.shape)`
- `print('Total number of values:',
score_table.size)`
- `print('Data type:', score_table.dtype)`

Score table:

```
[[72 80 75]  
 [85 78 82]  
 [64 70 68]  
 [90 88 92]  
 [58 60 62]]
```

Number of dimensions: 2

Shape: (5, 3)

Total number of values: 15

Data type: int64

numPy Array: Selecting Data



In NumPy, selecting data from an array can be done using: *Indexing, Slicing and Filtering*

Indexing: Indexing accesses a specific element using its position.

```
a = np.array([10, 20, 30, 40, 50])
```

```
print(a[0])
```

```
print(a[2])
```

In a 2-D array b:

```
print(b[0, 1])
```

numPy Array: Selecting Data



In NumPy, selecting data from an array can be done using: *Indexing, Slicing and Filtering*

Slicing: Slicing selects a range of elements - *Array[start:end]*

```
a = np.array([10, 20, 30, 40, 50])
```

```
print(a[1:4]) # 1 ≤ index < 4 -> [20 30 40]
```

```
print(a[:3]) # first 3 elements -> [10,20,30]
```

```
print(a[2:]) # from index 2 to end -> [30,40,50]
```

```
print(a[::2]) # step size 2 -> [10 30 50]
```

numPy Array: Selecting Data



In a 2-D array: same as 1-D, but slicing with both the dimensions

```
b = np.array([[1, 2, 3],  
              [4, 5, 6]])
```

```
print(b[:, 1:])
```

: → all rows

1: → columns from index 1 onward

numPy Array: Selecting Data



Filtering selects elements that satisfy a condition: Boolean Indexing

```
a = np.array([10, 20, 30, 40, 50])
```

```
print(a[a > 25]) -> O/P: [30 40 50]
```

```
Marks = np.array([72, 85, 64, 90, 58, 33, 49])
```

```
pass_mask = marks >= 50
```

```
print('Marks:', marks)
```

```
print('Pass mask:', pass_mask)
```

```
print('Passing marks only:', marks[pass_mask])
```

numPy Array: As vectors



- numPy arrays are vectors
- Allows performing operations on entire NumPy arrays without using explicit Python loops.
- Faster execution, Shorter code, Easier to read

```
import numpy as np
```

```
a = np.array([1, 2, 3, 4, 5])
```

```
result = a * 2
```

```
print(result) -> [ 2 4 6 8 10]
```

No loop is needed.

The multiplication happens element-wise:

$[1, 2, 3, 4, 5] \times 2$

numPy Array: As vectors - Element wise operations



```
- a = np.array([1, 2, 3])  
- b = np.array([4, 5, 6])  
- print(a + b)  
- print (a*b)
```

```
a = np.array([1, 4, 9, 16])  
  
print(np.sqrt(a))
```


numPy Array: As vectors - conditional selection using where()



- `np.where(condition, value_if_true, value_if_false)`

```
import numpy as np
```

```
a = np.array([10, 20, 30, 40, 50])
```

```
result = np.where(a > 25, 1, 0)
```

```
print(result)
```

Output: [0 0 1 1 1]

numPy Array: Other useful statistical functions



```
- marks = np.array([72, 85, 64, 90, 58, 33, 49])  
-  
- print('Marks:', marks)  
- print('Total:', np.sum(marks))  
- print('Average:', np.mean(marks))  
- print('Minimum:', np.min(marks))  
- print('Maximum:', np.max(marks))  
- print('Median:', np.median(marks))  
- print('Standard deviation:', np.std(marks))  
- print('Rounded standard deviation:', np.round(np.std(marks), 2))
```

numPy Array: Reshaping Arrays

- Reshaping changes the structure of an array without changing its data.
- Very useful for managing the feature vector to run an ML algorithm

NumPy provides:

- `reshape()` → convert array shape

```
a = np.array([1, 2, 3, 4, 5, 6])  
b = a.reshape(2, 3)
```

```
print(b)  
[[1 2 3]  
 [4 5 6]]
```

```
a.reshape(2, -1) // Only first dimension is given, second is automatically  
decided by python  
a.reshape(4, 2) // will raise an error! Why?
```

numPy Array: Reshaping Arrays

-  Flattening is very important in image processing, a gray scale image of A grayscale image:
- 28×28 will be flattened into: $28 \times 28 = 784$ elements vector as input to a neural network.

NumPy provides:

- `flatten()` → convert multi-dimensional array into 1D

```
b = np.array([[1, 2, 3],  
              [4, 5, 6]])
```

```
c = b.flatten() -> [1,2,3,4,5,6]  
Check shape of b and c
```



Thank you!